

02 · Speicherhierarchie & Caches

Mikrocomputertechnik 3 · Vom Lokalitätsprinzip bis zur Code-Optimierung

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 81

Willkommen zu Block 2!

- Willkommen zurück zur zweiten Vorlesung in Mikrocomputertechnik 3
- Rückblick: Block 1 hat Speicher auf Transistor-/Kondensatorebene untersucht (Flash, SRAM, DRAM)
- Heute: Bindeglied zwischen schneller CPU und langsamem RAM
- Fokus: das Konzept der Caches
- Ziel: verstehen, warum die Art Ihres C-Codes die Hardware massiv beeinflusst



Notizen

Herzlich willkommen zur zweiten Einheit! In Block 1 haben wir Speicher auf Transistor- und Kondensatorebene betrachtet (Flash, SRAM, DRAM). Heute verlassen wir die reine Physik und gehen zur Architektur: Caches verbergen die DRAM-Latenz. Dabei sehen Sie auch, warum die Struktur Ihres C-Codes die Hit Rate — und damit die Laufzeit — stark beeinflussen kann.

Folie

2 / 81

Anknüpfung: Memory Map und Linker

Kurz-Recap aus Block 1 (Labor folgt mit dem Übungsblatt):

- Memory Map: Flash, SRAM und MMIO belegen feste Adressbereiche
- Linker-Skript legt `.text` in den Flash, Variablen in den SRAM
- C-Pointer adressieren physische Bausteine über denselben Adressraum

```
// Prinzip aus Block 1:  
int global_var = 42;    // typisch: SRAM (.data)  
const int config = 1;  // typisch: Flash (.rodata)
```

Notizen

In Block 1 haben wir Memory Map und Linker-Skript konzeptionell eingeführt; das zugehörige Renode-Labor steht als Übungsblatt noch aus. Für heute reicht der Anschluss: Die CPU greift über Adressen auf SRAM/DRAM zu — und genau dort setzen Caches an.

Folie

3 / 81

Kurz-Recap: SRAM vs. DRAM

Aus Block 1 (Spickzettel) — die Motivation für Caches:

- SRAM (6T): extrem schnell, teuer, flächenintensiv → Caches/Register
- DRAM (1T1C): dicht und günstig, aber hohe Latenz (Refresh, Multiplexing)
- Wunsch „alles SRAM“ ist wirtschaftlich und flächenmäßig unrealistisch

	SRAM	DRAM
Geschwindigkeit	sehr hoch	niedriger
Kosten / Bit	hoch	niedrig
Typische Rolle	Cache	Hauptspeicher

Schematisch — Trendillustration aus Block 1.

Notizen

Kurz wiederholt aus Block 1: SRAM ist schnell, aber teuer; DRAM fasst Gigabytes, ist aber langsam. Genau dieses Spannungsfeld erzwingt die Speicherhierarchie und den Cache als Zwischenschicht.

Folie

4 / 81

Die CPU hungert — das Latenz-Problem

- Moderne CPU: ca. 3–5 GHz ($\approx 0,2\text{--}0,3$ ns pro Takt)
- DRAM-Latenz: ca. 50–100 ns
- Mismatch: Faktor $\approx 100\text{--}300$
- Folge: jedes $\perp$ w könnte hunderte Taktzyklen Stillstand bedeuten
- Pipeline aus MCT1 würde durch diesen Load-Use Hazard weitgehend entwertet

Schritt	Dauer (schematisch)
ALU Add	1 Takt
Warten auf DRAM	≈ 100 Takte
ALU Sub	1 Takt

Notizen

Hier ist unser fundamentales Problem. Unsere RISC-V Pipeline aus dem letzten Semester kann jeden Takt einen Befehl verarbeiten. Wenn wir jetzt einen Load-Befehl ausführen und auf den DRAM warten müssen, friert die gesamte CPU für 300 Taktzyklen ein. Das ist so, als ob Sie an Ihrem Schreibtisch arbeiten und für jedes einzelne Blatt Papier, das Sie lesen wollen, in den Keller ins Archiv laufen müssten.

Folie

5/81

Die Illusion erschaffen

- Ziel: Nutzer und CPU „betrügen“
- Illusion: Speicher so schnell wie SRAM und so billig/groß wie DRAM
- Trick: kleine, versteckte SRAM-Schicht zwischen CPU und DRAM
- Name: Cache (französisch: geheimes Versteck)
- Ideal: CPU merkt kaum, dass DRAM existiert

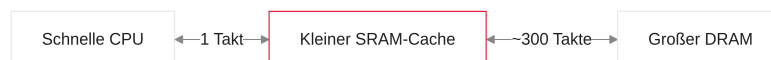
Notizen

Da wir die Physik nicht ändern können, bedienen wir uns einer Illusion. Wir geben der CPU einen winzigen, aber extrem schnellen Schreibtisch (den Cache). Den riesigen Aktenschrank (den DRAM) stellen wir in den Keller. Die Kunst besteht nun darin, immer genau die Akten auf den Schreibtisch zu legen, die die CPU in den nächsten Millisekunden brauchen wird. Wenn das klappt, rechnet die CPU mit maximaler Geschwindigkeit.

Folie

6/81

Cache als Zwischenschicht



Notizen

Schematisch: Die CPU spricht zuerst mit dem Cache. Nur bei einem Miss geht der Weg zum DRAM — oft hunderte Takte. Genau diese Zwischenschicht macht die Memory Wall handhabbar.

Agenda für den heutigen Tag

1. Memory Wall und Lokaliitätsprinzip — warum Caches funktionieren
2. Cache-Grundlagen und Terminologie (Hit, Miss, AMAT)
3. Cache-Mapping I: Direct Mapped
4. Cache-Mapping II: Set-Associative
5. Schreibstrategien: Write-Through vs. Write-Back
6. Brückenschlag zum C-Code; Ausblick Labor (z. B. Ripes)

Notizen

Um diese Illusion perfekt zu machen, müssen wir verstehen, wie der Cache organisiert ist. Wir starten mit der Theorie der Lokaliität, klären dann die Vokabeln der Computerarchitekten und schauen uns danach die konkreten Hardware-Schaltungen an, die Daten im Cache wiederfinden. Am Ende übersetzen wir das alles in C-Code.

Lernziele Block 2

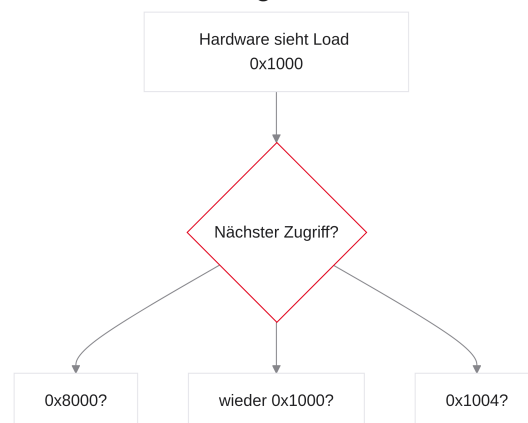
- Memory Wall erklären können
- Zeitliche und räumliche Lokaliität unterscheiden
- Hit Rate, Miss Penalty und AMAT berechnen
- Direct-Mapped und Set-Associative Caches unterscheiden
- C-Code so schreiben, dass Cache-Hits maximiert werden

Notizen

Am Ende dieser Vorlesung können Sie Memory Wall, Lokaliität und AMAT erklären sowie Direct-Mapped von Set-Associative unterscheiden. Cache-freundlichen C-Code werden Sie im geplanten Labor (z. B. mit Ripes) weiter vertiefen, sobald das Übungsblatt vorliegt.

Die Leitfrage des Tages

- Ein Cache ist nutzlos, wenn nicht die benötigten Daten darin liegen
- Hardware muss „ahnen“, was die Software als Nächstes tut
- Problem: Silizium kann keine C-Programme lesen und nicht in die Zukunft sehen
- Leitfrage: Wie errät eine dumme Schaltung mit $\approx 95\%$ Genauigkeit die nächsten Adressen?
- Die Antwort hat die moderne Informatik gerettet



Notizen

Das ist das zentrale Rätsel der heutigen Vorlesung. Wenn ich in den Keller laufe, um eine Akte zu holen, muss ich wissen, welche Akten ich gleich mit nach oben nehmen soll, sonst laufe ich ständig Treppen. Aber der Cache ist eine simple Hardware-Schaltung. Er hat keine künstliche Intelligenz. Er sieht nur nackte Adressen aufklicken. Wie schafft er es trotzdem, fast immer das Richtige bereitzuhalten?

Die Memory Wall im Detail

- Seit ~1980: CPU-Geschwindigkeit wuchs jahrzehntelang stark
- DRAM: deutlich langsames Wachstum der Latenz/Performance
- Diese öffnende Schere heißt Memory Wall (Speichermauer)
- Ab einem Punkt bringt ein schnellerer Kern kaum noch Zuwachs
- Flaschenhals: Datenlieferung, nicht die ALU

Epoche (schematisch)	CPU relativ	DRAM relativ
1980	1×	1×
2000	$\approx 100\times$	$\approx 4\times$
2020	$\approx 10\,000\times$	$\approx 16\times$

Notizen

Wir haben es bereits kurz in Block 1 angeschnitten, aber hier ist die dramatische Entwicklung. Architekten haben die Pipelines immer tiefer gemacht und die Taktraten erhöht. Aber der DRAM-Speicher ist auf seinem Kondensator-Prinzip sitzen geblieben. Die Latenz sank kaum. Irgendwann in den 1990er Jahren traf die CPU-Entwicklung auf eine Wand. Eine schnellere CPU brachte 0 % mehr Leistung bei speicherlastigen Programmen.

Folie

11 / 81

Zyklen vs. Zugriffszeiten

Beispiel: RISC-V-SoC mit 3 GHz

- 1 Takt: $\frac{1}{3 \cdot 10^9} \text{ s} \approx 0,33 \text{ ns}$
- L1-Cache: ca. 1 ns $\rightarrow \approx 3$ Takte
- DRAM: ca. 100 ns $\rightarrow \approx 300$ Takte
- Jeder Hauptspeicherzugriff kostet hunderte Arbeitsgelegenheiten

Notizen

Zahlen sprechen lauter als Worte. Bei 3 Gigahertz tickt die Uhr der CPU alle 0,33 Nanosekunden. Wenn wir auf den L1-Cache zugreifen, geht das rasend schnell, vielleicht verlieren wir ein oder zwei Takte. Aber wenn wir auf den RAM warten, stehen wir 300 Takte still! Stellen Sie sich vor, Sie müssten bei jedem Wort, das Sie lesen, 300 Sekunden (5 Minuten) die Augen schließen. Sie würden niemals ein Buch beenden.

Folie

12 / 81

Stalls und Bubbles — Link zu MCT1

- Während des Wartens greift die Hazard Detection Unit ein
- Massiver Load-Use Hazard
- Fetch- und Decode-Stufen frieren ein
- Hardware schiebt Bubbles (NOPs) in Execute
- Teurer Prozessor: Strom ohne Nutzarbeit

```
lw  t0, 0(s1)      # startet lange Wartezeit
# --- hier entstehen hunderte Bubbles ---
add t1, t0, t2      # muss auf Daten warten
```

Notizen

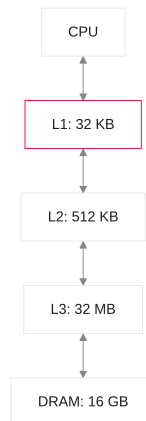
Schlagen wir die Brücke zu unserem Wissen aus MCT1. Wir kennen die Konsequenz aus Pipelining-Sicht. Das ist ein extremer Load-Use Hazard. Die Hazard Unit muss die Pipeline anhalten (Stall) und wirft hunderte von leeren Blasen (NOPs) in das Fließband. Die Kühlung läuft, der Stromzähler dreht sich, aber es werden keine Berechnungen durchgeführt.

Folie

13/81

Die Lösung: Speicherhierarchie

- L1: winzig (z. B. 32 KB), on-chip SRAM, 1–3 Takte
- L2: größer (z. B. 512 KB), SRAM, 10–15 Takte
- L3: sehr groß (z. B. 32 MB), SRAM, 30–40 Takte
- Main Memory: riesig (z. B. 16 GB), DRAM, ≈ 300 Takte
- Hardware kopiert Blöcke automatisch zwischen den Schichten



Notizen

Da wir das physikalische Dreieck (schnell, groß, billig) nicht besiegen können, kombinieren wir alles. Wir bauen Caches in mehreren Leveln. Fällt das gesuchte Datum nicht im L1-Cache an, fragt dieser den L2. Fällt es dort nicht an, fragt der den L3, und erst dann geht es zum DRAM. Das Geniale daran: Sie als Software-Entwickler müssen diese Kopiervorgänge nicht programmieren. Die Hardware macht das völlig unsichtbar im Hintergrund.

Die Geheimwaffe: Lokalitätsprinzip

- Warum funktioniert die Pyramide so gut?
- Software zeigt kein rein zufälliges Zugriffsverhalten
- Code hat Struktur: Schleifen, Arrays
- Principle of Locality (Lokalitätsprinzip)
- Zwei Säulen: zeitliche und räumliche Lokalität

Notizen

Jetzt lösen wir unsere Leitfrage auf. Warum funktioniert der Cache? Weil unsere Programme nicht chaotisch auf Speicher zugreifen. Wenn Programme bei jedem Takt eine völlig zufällige Adresse aus 16 Gigabyte RAM abfragen würden, wäre der Cache nutzlos und die Hit-Rate läge bei 0 %. Aber Code wird von Menschen geschrieben. Wir nutzen Schleifen. Wir nutzen strukturierte Arrays. Dieses geordnete Verhalten rettet die Architektur.

Säule 1: Zeitliche Lokalität

- Temporal Locality: eine gerade genutzte Adresse wird sehr wahrscheinlich bald wieder genutzt
- Metapher: aufgeschlagenes Buch bleibt auf dem Schreibtisch
- Hardware: geladene Daten bleiben eine Weile im Cache

Notizen

Die erste Säule ist die zeitliche Lokalität. Die Regel lautet: Was gerade benutzt wurde, wird bald wieder gebraucht. Das klingt banal, ist aber die fundamentale Daseinsberechtigung für den Cache. Wenn die CPU eine Adresse anfragt, kopiert der Cache sie nicht nur durch, sondern speichert sie ab, weil er darauf wettet, dass die CPU bald wieder genau dieselbe Adresse anfragen wird.

Zeitliche Lokalität im C-Code

- Schleifenzähler `i` und Grenze `limit`: in jedem Durchlauf gelesen
- Auch die Befehle der Schleife werden hundertfach erneut geladen
- Nach dem ersten Durchlauf: Hits aus dem L1-SRAM

```
int limit = 1000;
int sum = 0;

for (int i = 0; i < limit; i++) {
    // i, sum und limit: hohe zeitliche Lokalität
    sum += i;
}
```

Notizen

Schauen Sie sich diesen simplen C-Code an. In dieser For-Schleife wird die Variable `i` tausendmal gelesen und geschrieben. Die Variable `limit` wird tausendmal gelesen. Die Maschinenbefehle der Schleife werden tausendmal ausgeführt. Der Cache muss also nur beim allerersten Durchlauf auf den langsamen DRAM warten. Für die restlichen 999 Durchläufe liegen die Variablen und die Befehle bereits im L1-Cache bereit!

Säule 2: Räumliche Lokalität

- Spatial Locality: nach einer Adresse folgen sehr wahrscheinlich die Nachbarn
- Metapher: Band 2 der Enzyklopädie — Band 3 gleich mitnehmen
- Hardware: nie nur ein Byte — immer ein ganzer Block (Cache Line)

Notizen

Die zweite Säule ist die räumliche Lokalität. Die Wette der Hardware lautet hier: Wenn die CPU die Adresse 1000 liest, wird sie als Nächstes sehr wahrscheinlich die Adresse 1004 lesen. Um Zeit zu sparen, bringt der Cache beim Besuch im DRAM nicht nur das angefragte Byte mit, sondern packt gleich die nächsten 64 Bytes mit ein. Das ist ein Geniestreich!

Räumliche Lokalität im C-Code

Zwei typische Quellen:

1. Sequenzielle Befehle: nach Befehl n oft Befehl $n + 1$
2. Arrays und Strings: benachbarte Elemente hintereinander

Wenn `array[0]` geladen wird, lädt der Cache oft schon `array[1]`, `[2]`, ...

```
int daten[100];

// Klassischer Array-Durchlauf - hohe räumliche Lokalität
for (int i = 0; i < 100; i++) {
    daten[i] = 0;
}
```

Notizen

Warum stimmt diese Wette der Hardware fast immer? Weil Code sequenziell ist. Der Program Counter zählt fast immer einfach +4 weiter. Und bei Datenstrukturen lieben wir Arrays. Wenn Sie in C ein Array anlegen, liegen die Elemente im RAM physikalisch direkt nebeneinander. Wenn Sie das erste Element lesen, zieht der Cache den gesamten restlichen Datenblock in den schnellen SRAM. Der Zugriff auf Element 2, 3 und 4 kostet dann in der Regel nur noch die Hit Time — ohne erneute Miss Penalty.

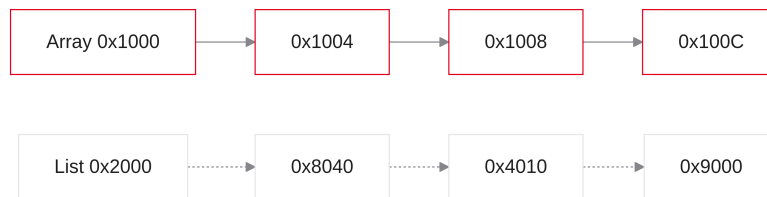
Array vs. Linked List

- Pointer-Ketten: Knoten oft per `malloc()` im Speicher verstreut
- Nachbarn liegen physikalisch oft Megabytes auseinander
- Folge: räumliche Lokalität bricht zusammen
- Cache lädt Blöcke — Nachbardaten sind für die Liste oft nutzlos
- Linked Lists erzeugen viele Cache Misses; Arrays sind cache-freundlicher

Notizen

Das ist eine fundamentale Erkenntnis für Sie als Software-Entwickler. In der Informatik lernt man oft, dass verkettete Listen wegen flexibler Einfügeoperationen attraktiv sind. Aus Sicht der modernen Hardware sind sie problematisch: Die Knoten liegen quer im RAM verstreut. Der Cache lädt bei Knoten 1 oft Daten mit, die für Knoten 2 nutzlos sind. Arrays sind auf modernen CPUs deshalb oft um ein Vielfaches schneller — allein wegen besserer Cache-Nutzung.

Layout im Speicher: Array vs. Liste



Notizen

Oben: Array-Elemente liegen physikalisch hintereinander — ideal für räumliche Lokalität. Unten: verkettete Liste mit Streuung im Adressraum — der Cache kann Nachbarn kaum sinnvoll mitnutzen.

Die 90/10-Regel

- Faustregel: $\approx 90\%$ der Ausführungszeit in $\approx 10\%$ des Codes
- Typisch: innere Schleifen (Rendering, Matrizen, ...)
- Dieser Hotspot passt oft vollständig in den kleinen L1-Cache
- Deshalb kann 32 KB Cache ein 16-GB-DRAM-System massiv beschleunigen

Notizen

Diese empirische Regel rettet die Architektur. Sie schreiben vielleicht ein Programm mit Millionen Zeilen Code, aber der Computer rechnet die meiste Zeit in einer winzigen For-Schleife tief im Algorithmus. Dieser winzige Hotspot passt perfekt in unseren winzigen L1-Cache. Der restliche Code stört nicht, er wird nur einmal beim Start geladen.

Fragen? — Lokalität und Memory Wall

- Fragen zur Memory Wall?
- Zeitliche vs. räumliche Lokalität klar?
- Warum sind Arrays cache-freundlicher als Linked Lists?

Notizen

Bevor wir die konkreten Vokabeln und Formeln klären: Sind diese Grundprinzipien klar? Die gesamte Existenzberechtigung von Caches basiert auf der Annahme, dass Software sich vorhersagbar verhält.

Was ist ein Cache?

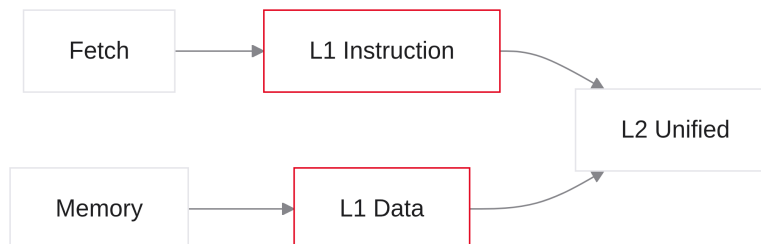
- Kleiner, schneller Speicher mit Kopien aus einem größeren, langsameren Speicher
- Transparent: Software sieht ihn in der Regel nicht (`load_to_cache` gibt es nicht)
- Steuerwerk kopiert, sucht und verwaltet autonom
- CPU fordert `0x1000` an — der Cache fängt die Anfrage ab

Notizen

Wir klären nun das Vokabular. Wichtig ist: Der Cache ist unsichtbar. In MCT1 haben wir Register mit `lw` und `sw` manuell geladen. Den Cache laden Sie nicht manuell. Wenn Sie einen Load-Befehl ausführen, prüft die Hardware heimlich, ob die Adresse im Cache liegt. Wenn ja, antwortet der Cache. Wenn nein, reicht der Cache die Anfrage an den RAM weiter.

Split Caches: L1i und L1d

- Von-Neumann-Flaschenhals auf L1-Ebene entschärfen
- Split Caches (Harvard auf L1):
 - L1-I: nur Befehle (Fetch)
 - L1-D: nur Daten (Memory)
- Vorteil: Fetch und Memory gleichzeitig, ohne Streit um denselben Port
- L2 oft unified (gemeinsam für Code und Daten)



Notizen

Auf dem Level 1, also direkt an der CPU, haben wir fast immer zwei Caches. Einen für Code, einen für Daten. Warum? Damit unsere Pipeline aus MCT1 nicht blockiert. Wenn die Fetch-Stufe den nächsten Befehl laden will und die Memory-Stufe gleichzeitig ein Array liest, würden sie sich bei einem gemeinsamen Cache blockieren. Durch Split-Caches haben wir zwei getrennte Eingangstüren. Auf Level 2 wird dann meistens alles wieder in einem gemeinsamen (unified) Cache zusammengefasst.

Cache Hit vs. Cache Miss

Cache Hit (Treffer):

- Adresse liegt als Kopie im Cache
- Lieferung in 1–3 Takten

Cache Miss (Fehlschlag):

- Daten nicht im Cache
- Weiterleitung zum Hauptspeicher (z. B. ≈ 300 Takte)

Notizen

Das sind die wichtigsten Vokabeln des heutigen Tages. Ein Cache Hit ist ein Treffer, ein Erfolg. Die Pipeline rennt weiter. Ein Cache Miss ist ein Fehlschlag. Der Cache muss zugeben, dass er das gesuchte Datum nicht hat. Jetzt friert die CPU ein und wir müssen auf den lahmen Hauptspeicher warten.

Hit Rate und Miss Rate

$$\text{Hit Rate} = \frac{\text{Hits}}{\text{Gesamtzugriffe}}, \quad \text{Miss Rate} = 1 - \text{Hit Rate}$$

- Hit Rate: Anteil der Zugriffe, die im Cache gefunden werden
- Miss Rate: Anteil der Fehlschläge
- Praxis: gute L1-Caches oft 90–98 % Hit Rate

Notizen

Die Hit Rate ist die wichtigste Kenngröße. Wenn Sie ein Programm analysieren und eine Hit Rate von 95 % sehen, bedeutet das, dass 95 von 100 Speicherzugriffen sofort vom Cache bedient werden. Nur bei 5 % müssen wir in den RAM greifen. Das klingt hervorragend, aber wir werden gleich mathematisch sehen, dass selbst 5 % Fehler extrem weh tun können.

Miss Penalty

- Hit Time: Zeit für Treffer-Erkennung und Lieferung (z. B. 1 ns)
- Miss Penalty: zusätzliche Zeit bei Fehlschlag bis DRAM (z. B. 100 ns)
- Bei schlechter Hit Rate dominiert die Penalty die Laufzeit

Notizen

Die Strafzeit ist das, was uns wehtut. Wenn wir raten und danebenliegen, verlieren wir Zeit. Wir müssen erst feststellen, dass die Daten nicht da sind (das dauert die Hit Time), und dann müssen wir den langen Weg zum RAM antreten (Miss Penalty). Die Penalty ist oft um den Faktor 100 größer als die Hit Time.

AMAT — Average Memory Access Time

Klausurrelevante Kernformel:

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

- Jeder Zugriff kostet mindestens die Hit Time (Cache wird immer geprüft)
- Nur bei Miss kommt die gewichtete Penalty hinzu

Notizen

Das ist die wichtigste Formel der Speicherarchitektur. Wenn Sie bei Intel oder Apple Chips designen, optimieren Sie diese Formel. Die Average Memory Access Time sagt uns, wie lange die CPU im Schnitt für einen Speicherzugriff warten muss. Wir zahlen immer den Preis für den Blick in den Cache (Hit Time). Und in einem bestimmten Prozentsatz der Fälle (Miss Rate) zahlen wir zusätzlich die teure Strafe (Miss Penalty).

Rechenbeispiel: Macht des Caches

- L1 Hit Time: 1 Takt
- RAM Miss Penalty: 100 Takte

Ohne Cache: AMAT = 100 Takte.

Mit Cache, Hit Rate 95 % (Miss Rate 5 %):

$$\text{AMAT} = 1 + (0,05 \times 100) = 6 \text{ Takte}$$

Fazit: DRAM fühlt sich für die CPU wie ≈ 6 Takte an — massive Beschleunigung.

Notizen

Lassen Sie uns das rechnen. Wenn wir keinen Cache hätten, würden wir im Schnitt 100 Takte pro Zugriff warten. Mit unserem Cache und einer Hit Rate von 95 % sieht die Welt völlig anders aus. Wir warten 1 Takt plus 5 % von 100 Takten. Das ergibt 6 Takte! Der Cache hat unseren extrem langsamen PC gefühlt um den Faktor 16 beschleunigt. Und das nur, weil Software das Lokalitätsprinzip befolgt.

Wir laden niemals einzelne Bytes

- Räumliche Lokalität $\Rightarrow$ keine Einzelbyte-Verwaltung
- Verwaltungseinheit: Cache Block / Cache Line
- Typisch 32 oder 64 Bytes zusammenhängender Speicher
- Ein Miss lädt immer die gesamte Line aus dem DRAM

Offset	0	1	2	3	...	63
Inhalt	Byte	Byte	Byte	Byte	...	Byte

Eine Cache Line = ein zusammenhängender Block (hier 64 Bytes, schematisch).

Notizen

Jetzt gehen wir ins Hardware-Design. Wenn die CPU ein einziges Byte auf Adresse 1000 anfragt und der Cache dieses Byte nicht hat, dann lädt der Cache nicht etwa nur dieses eine Byte aus dem RAM. Er lädt einen Block, z. B. die Adressen 1000 bis 1063 auf einmal! Dieser Block nennt sich Cache Line.

Cache Line und räumliche Lokalität

- Miss bei `array[0]` lädt die ganze Line (z. B. 16 Integers)
- `array[1]` ... bis zum Line-Ende: garantierte Hits
- Ein Miss erkaufte viele Hits über Nachbarschaft

Notizen

Dieses Konzept der Cache Lines implementiert die räumliche Lokalität in Silizium. Der Miss beim ersten Array-Element schmerzt, wir warten 100 Takte. Aber der Speicher-Controller zieht heimlich die nächsten 15 Elemente mit nach oben. Die nächsten 15 Schleifendurchläufe sind also reine Hits, die jeweils nur 1 Takt brauchen! Ohne Cache Lines müssten wir für jedes Element den vollen Preis zahlen.

Anatomie einer Cache-Zeile

Ein Cache-Eintrag enthält drei Felder:

1. Daten (Block): z. B. 64-Byte-Paket aus dem RAM
2. Valid-Bit: 1 = gültige Daten, 0 = leer/ungültig (nach Reset)
3. Tag: Adress-Etikett — woher aus dem großen Adressraum stammt der Block?

Valid	Tag	Daten
1 Bit	z. B. 17 Bit	z. B. 64 Bytes

Notizen

Im SRAM des Caches speichern wir nicht nur die Daten. Wir müssen die Daten markieren. Stellen Sie sich vor, Sie legen Aktenordner aus dem Keller auf den Schreibtisch. Wenn auf dem Ordner nicht steht, wo er herkommt, wissen Sie später nicht, wem die Daten gehören. Dieses Etikett ist das Tag. Und das Valid-Bit brauchen wir, weil der Cache beim Einschalten des PCs voller Strom-Rauschen und Müll ist. Das Valid-Bit sagt: Diese Daten sind frisch und echt.

Blockgröße: Der Kompromiss

- Größere Lines: bessere räumliche Lokalität — aber höhere Miss Penalty
- Wenige große Blöcke: weniger „Slots“, mehr Verdrängung (Thrashing)
- Industrie-Sweetspot: oft 32–64 Bytes pro Line

Notizen

Könnten wir nicht einfach 4-Kilobyte-Blöcke laden? Die Antwort liegt wieder im magischen Dreieck. Ein L1-Cache hat vielleicht nur 32 Kilobyte. Wenn wir 4 Kilobyte große Blöcke nutzen, passen genau 8 Blöcke in den Cache. Wenn das Programm nun an 9 verschiedenen Stellen im RAM gleichzeitig arbeitet, werfen sich diese Blöcke ständig gegenseitig aus dem Cache. Die Miss Penalty explodiert, weil das ständige Nachladen den Speicherbus blockiert. 64 Bytes haben sich heute als guter Kompromiss erwiesen.

Fragen? — Metriken und Cache Lines

- AMAT-Berechnung nachvollziehbar?
- Warum ist eine Cache Line größer als ein Maschinenwort (4 Bytes)?
- Zweck von Valid-Bit und Tag?

Notizen

Wir haben die Metriken und den logischen Aufbau einer Cache-Line besprochen. Im nächsten Schritt werden wir in die Schaltpläne eintauchen und klären, wie die Hardware blitzschnell herausfindet, in welcher der hundert Cache-Zeilen unsere gesuchten Daten liegen. Gibt es bis hierhin noch Fragen?

Das Mapping-Problem

- Massives Platzproblem zwischen RAM und Cache
- RAM: z. B. 4 GB (Millionen Cache-Blöcke)
- L1-Cache: z. B. 32 KB (vielleicht nur 512 Cache-Zeilen)
- Frage: Wo darf ein RAM-Block im Cache abgelegt werden?
- Herausforderung: Mapping in Hardware, idealerweise in höchstens 1 Takt
- Drei grundlegende Strategien lösen dieses Problem

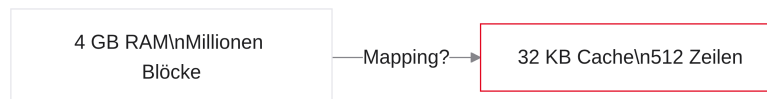
Notizen

Wir stehen vor einem logistischen Engpass. Wir haben Millionen von Aktenordnern im Keller, aber nur 512 freie Plätze auf unserem Schreibtisch. Wenn wir eine Akte hochholen, wo legen wir sie ab? Und wenn die CPU eine Akte sucht, wie findet sie diese in etwa 1 Nanosekunde wieder? Wenn die CPU jede der 512 Positionen nacheinander durchsuchen müsste, wäre der Zeitvorteil des Caches sofort wieder zerstört.

Folie

36 / 81

Mapping: Kapazität vs. Suche



Notizen

Schematische Größenordnung: Der Hauptspeicher fasst Millionen Blöcke, der L1-Cache nur wenige hundert Zeilen. Die Mapping-Funktion muss deshalb in Hardware extrem schnell entscheiden, wohin ein Block gehört und wo er gesucht wird.

Folie

37 / 81

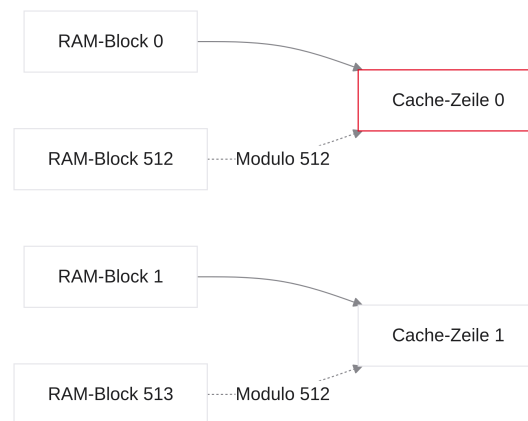
Direct Mapped Cache

- Einfachste Strategie: Direct Mapped (Direktabbildung)
- Regel: jeder RAM-Block hat genau einen festen Platz im Cache
- Parkhaus-Metapher: Parkplatznummer aus den letzten Ziffern des Nummernschilds
- Endet das Schild auf 05 → nur Parkplatz 05
- Vorteil: Suche nur an genau diesem einen Platz

Notizen

Direct Mapped ist die simpelste und schnellste Hardware-Lösung. Jedem RAM-Block wird mathematisch ein fester Platz im Cache diktiert. Das ist wie in einem Parkhaus, in dem Sie nicht frei parken dürfen. Wenn die CPU wissen will, ob Block 512 im Cache ist, weiß sie sofort: „Wenn er da ist, dann zwingend auf Platz 0.“ Die Hardware schaut nur auf Fach 0. Ist er da, Hit. Ist er nicht da, Miss.

Direct Mapped: feste Zuordnung



Notizen

Beispiel mit 512 Zeilen: Block 0 und Block 512 konkurrieren um dieselbe Cache-Zeile 0. Genau diese Kollision führt später zum Thrashing-Problem.

Modulo-Arithmetik in Hardware

- $\text{Cache-Index} = (\text{RAM-Blockadresse}) \bmod (\text{Anzahl Cache-Zeilen})$
- Echter Dividierer wäre zu langsam
- Trick: Caches in Zweierpotenzen (z. B. $512 = 2^9$ Zeilen)
- Modulo durch Zweierpotenz = Abschneiden der oberen Bits (Bit-Maskierung)

Notizen

Wie bringt man Silizium dazu, Modulo zu rechnen, ohne einen riesigen Taschenrechner (ALU) anzuwerfen? Ganz einfach: durch reines Verdrahten von Kupferleitungen. Wenn Sie modulo 100 im Dezimalsystem rechnen, nehmen Sie einfach die letzten zwei Ziffern (1234 modulo 100 ist 34). In der Binärwelt schneiden Sie einfach Leitungen ab. Das kostet exakt 0 Taktzyklen.

Die Adress-Zerlegung

Die CPU legt eine 32-Bit-Adresse auf den Bus. Der Cache zerschneidet sie in drei Felder:

1. Block Offset — unterste Bits
2. Cache Index — mittlere Bits
3. Tag — oberste Bits

Diese drei Felder sind der Schlüssel jeder Cache-Architektur.

Tag (z. B. 17 Bit)	Index (z. B. 9 Bit)	Offset (z. B. 6 Bit)
oberste Bits	mittlere Bits	unterste Bits

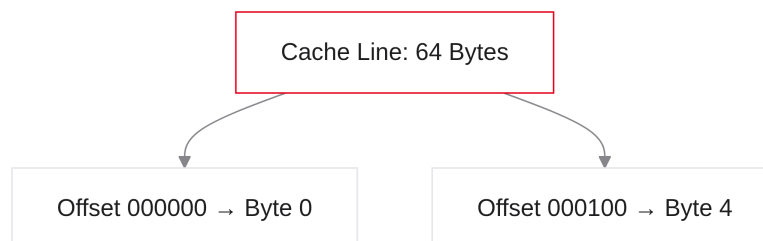
Notizen

Das ist das Rezept der Hardware. Die 32-Bit-Speicheradresse ist nicht nur eine Hausnummer, sie ist ein Wegweiser. Der Cache nimmt die 32 Kupferkabel der Adresse und teilt sie in drei Bündel auf. Das ist reine Kabelführung, es gibt hier keine logischen Gatter, die verzögern. Wir schauen uns jetzt diese drei Bündel im Detail an.

Folie

Feld 1: Der Block Offset

- Wo in der Cache-Zeile liegt das gesuchte Byte?
- Eine Line bringt z. B. 64 Bytes am Stück
- CPU will oft nur ein 4-Byte-Wort
- $2^6 = 64 \rightarrow$ unterste 6 Bits als Offset innerhalb des Blocks



Notizen

Das erste Bündel sind die untersten Bits. Erinnern wir uns: Der Cache lädt immer fette 64-Byte-Blöcke. Wenn der 64-Byte-Block endlich gefunden ist, müssen wir aus diesen 64 Bytes das richtige Stück herausausschneiden, das die CPU angefragt hat. Bei 64 Bytes brauchen wir exakt 6 Bits, um jedes Byte darin durchnummerieren.

Feld 2: Der Cache Index

- Welche Zeile des Caches? (Modulo-Operation)
- Mittlere Bits schneiden die Adresse auf die Cache-Größe
- Beispiel: 512 Zeilen $\Rightarrow 2^9 \rightarrow$ nächste 9 Bits über dem Offset
- Diese Bits gehen direkt an den Adress-Decoder des Cache-SRAM

Notizen

Das ist der Index. Er sagt der Hardware, in welche Schublade sie greifen muss. Wenn wir 512 Schubladen haben, reichen 9 Bits völlig aus. Wir nehmen also die nächsten 9 Kabel der Adresse, schließen sie an den SRAM des Caches an und die Schublade springt sofort auf. Kein Suchen, kein Warten. Purer Random Access in Sub-Nanosekunden.

Feld 3: Das Tag

- Prüft: Sind das wirklich meine Daten?
- Alias-Problem: Millionen RAM-Blöcke teilen denselben Index
- Beim Öffnen der Zeile: welches der vielen Blöcke liegt darin?
- Oberste Bits = Tag
- Komparator: CPU-Tag vs. gespeichertes Tag im Cache-SRAM

Notizen

Hier kommt das Tag ins Spiel. Da wir nur 512 Plätze haben, müssen sich sehr viele RAM-Blöcke denselben Parkplatz teilen. Block 0 parkt auf Platz 0. Aber Block 512 parkt auch auf Platz 0. Wenn die CPU Platz 0 öffnet, muss sie auf das Etikett (das Tag) schauen. Steht auf dem Etikett „0“? Oder steht da „512“? Ein riesiges XOR-Gatter vergleicht das CPU-Tag mit dem Cache-Tag. Sind sie gleich, haben wir einen Hit!

Das Valid-Bit — Boot-Problem

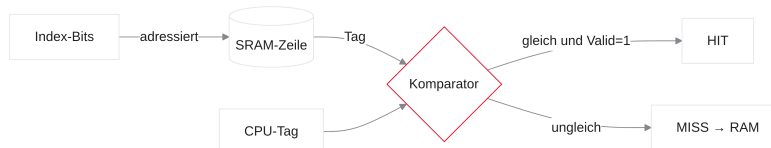
- Nach Einschalten: SRAM voller zufälligem Rauschen
- Zufällig könnte das Rauschen dem gesuchten Tag gleichen
- Lösung: 1-Bit Valid-Flag
- Hardware-Reset setzt alle Valid-Bits auf 0
- Hit nur wenn Tag übereinstimmt und Valid = 1

Notizen

Das Valid-Bit rettet den Computer beim Einschalten. Ohne dieses Bit würde die CPU beim ersten Load-Befehl einen Hit melden, weil der physikalische Zufallsmüll im SRAM zufällig das richtige Tag-Muster aufweist. Das Valid-Bit sagt knallhart: „Dieser Eintrag ist Müll, er wurde noch nie absichtlich aus dem RAM geladen.“

Ablauf eines Lesezugriffs (Direct Mapped)

1. 32-Bit-Adresse anliegend — Index-Bits öffnen das Cache-SRAM
2. Gespeichertes Tag und Valid-Bit auslesen
3. Komparator: $(\text{Tag} = \text{CPU-Tag}) \wedge (\text{Valid} = 1)$
4. Hit → Daten zur CPU; Miss → Stall und Anfrage an Main Memory



Notizen

Das ist der komplette logische Fluss eines Direct Mapped Caches. Alles passiert völlig parallel. Der Index öffnet den SRAM, der Komparator vergleicht die Tags, und der MUX bereitet schon mal die Daten vor. Wenn der Komparator „Hit“ meldet, wird die Tür zur CPU aufgeschoben.

Vorteil von Direct Mapped

- Suche nur an einer Stelle → minimaler Hardware-Aufwand
- Nur ein Komparator: Strom und Chipfläche sparen
- Schnelle Hit-Time: oft Antwort in einem Takt
- Bei Miss keine Wahl: nur ein Platz kann verdrängt werden

Notizen

Direct Mapped Caches sind wunderschön simpel. Die Hardware-Ingenieure lieben sie, weil sie kaum Strom verbrauchen und rasend schnell sind. Es gibt keine Diskussion, wer den Parkplatz räumen muss, wenn ein neues Auto kommt. Es gibt für das neue Auto ja eh nur diesen einen Platz. Der alte Besitzer fliegt gnadenlos raus.

Nachteil: Thrashing

- Starrheit hat einen Preis: Thrashing (zerstörerisches Flattern)
- Szenario: abwechselnder Zugriff auf Variable A und B (z. B. $A = A + B$)
- A auf Block 0, B auf Block 512 → gleicher Cache-Index 0
- Folge: A wirft B raus, B wirft A raus — Ping-Pong
- Cache zu 99 % leer, Miss Rate trotzdem $\approx 100\%$

Notizen

Hier scheitert der Direct Mapped Cache brutal. Wenn zwei Variablen, die Sie im C-Code ständig abwechselnd nutzen, zufällig denselben Index im Cache haben, zerstören sie sich gegenseitig. Das nennt man Thrashing. Der Cache wirft ständig die eine Variable weg, um Platz für die andere zu machen. Der Rest des Caches bleibt komplett ungenutzt, aber Ihr Programm steht praktisch still, weil es bei jedem Zugriff auf den lahmen RAM warten muss.

Thrashing-Beispiel in C

Thrashing *kann* auftreten, wenn zwei heiß genutzte Arrays denselben Cache-Index treffen (abhängig von Alignment, Array-Größe und Cache-Organisation):

```
// Kollision möglich bei gleichem Cache-Index
int A[1000];
int B[1000];

for (int i = 0; i < 1000; i++) {
    // Konflikt: gegenseitiges Verdrängen
    A[i] = A[i] + B[i];
}
```

Notizen

Ein harmlos wirkendes C-Programm: Zwei Arrays werden addiert. Wenn der Linker diese Arrays so platziert, dass ihre Adressen denselben Cache-Index ergeben — und der Cache Direct Mapped ist — können sich A und B gegenseitig verdrängen. Dann läuft das Programm deutlich langsamer. Ob die Kollision wirklich eintritt, hängt von Alignment, Größen und Cache-Parametern ab; dem Quelltext allein sieht man das oft nicht an.

Fragen? — Direct Mapped

- Direct Mapped verstanden?
- Zerlegung in Tag, Index und Offset klar?
- Warum kann Thrashing auftreten?

Notizen

Gibt es Fragen zur Adresszerlegung? Das ist ein beliebtes Klausurthema. Wenn wir das Problem des Thrashings verstanden haben, schauen wir uns jetzt an, wie Hardware-Architekten dieses Problem in den Griff bekommen haben.

Fully Associative Cache

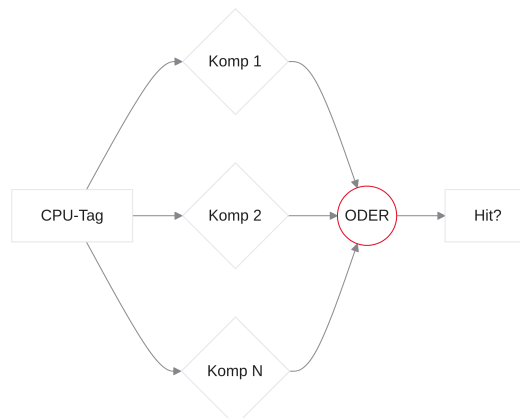
- Gegenteil von Direct Mapped: Fully Associative (vollassoziativ)
- Regel: RAM-Block darf an jedem freien Platz liegen
- Parkhaus: freie Platzwahl
- Vorteil: kein Thrashing — A und B können nebeneinander leben

Notizen

Um Thrashing zu stoppen, geben wir die starren Parkplätze auf. Fully Associative heißt absolute Freiheit. Ein Block aus dem RAM darf in jede beliebige Zeile des Caches geschrieben werden. Dadurch können Variable A und Variable B aus unserem C-Beispiel friedlich nebeneinander im Cache leben, auch wenn ihre Adressen eigentlich kollidieren würden.

Das Such-Problem bei Fully Associative

- Freiheit hat einen Preis: Wo liegt der Block?
- Kein Index mehr — Adresse nur noch Tag und Offset
- Hardware muss alle Zeilen parallel nach dem Tag durchsuchen
- Bei 512 Zeilen: 512 parallele Komparatoren



Notizen

Wenn es keinen festen Parkplatz mehr gibt, muss die CPU jedes Mal das gesamte Parkhaus absuchen, um ihr Auto zu finden. In der Informatik lösen wir das durch massive Parallelität. Wir bauen für jede einzelne Zeile des Caches einen eigenen Komparator-Schaltkreis. Wenn die CPU eine Anfrage stellt, brüllen alle 512 Komparatoren gleichzeitig auf. Einer meldet hoffentlich: „Ich hab's!“.

Nachteile von Fully Associative

- Strom und Hitze: hunderte parallele Vergleiche jede Nanosekunde
- Chipfläche: Komparatoren oft größer als der SRAM selbst
- Latenz: große ODER-Netze verlangsamen das Hit-Signal
- Fazit: für große L1-Caches in der Praxis unbrauchbar

Notizen

Fully Associative Caches sind theoretisch attraktiv, weil sie Hit-Raten maximieren — physikalisch aber teuer: Hitzeentwicklung und Platzbedarf der parallelen Vergleichslogik machen die Architektur für große L1-Caches ungeeignet. Wir brauchen einen Kompromiss aus beiden Welten.

Der Mittelweg: Set-Associative Cache

- Industriestandard: Set-Associative (mengenassoziativ)
- Hybrid aus Direct Mapped und Fully Associative
- Parkhaus: feste Zone (Set) wie Direct Mapped — darin z. B. 4 freie Plätze (Ways)
- Sie müssen in Zone 05 parken, dürfen dort aber einen der 4 Plätze wählen

Notizen

Willkommen beim Industriestandard. Fast jeder Prozessor in Ihrem Handy oder Laptop nutzt Set-Associative Caches. Wir mischen die Strategien. Die Adresse sagt uns wieder: „Geh zu Index 5“. Aber auf Index 5 gibt es nicht nur eine Cache-Zeile, sondern eine kleine Menge (ein Set) von Zeilen.

N-Way Set-Associative Struktur

- Cache in Sets; jedes Set hat N Ways
- Beispiel 4-Way: Index zeigt auf Set mit 4 Blöcken
- Suche: nur N parallele Komparatoren (z. B. 4)
- Bestes aus beiden Welten:
 - wenig Komparatoren (schnell, sparsam)
 - bis zu N Blöcke mit gleichem Index ohne gegenseitiges Rauswerfen

Index	Way 0	Way 1	Way 2	Way 3
Set 0	Block	Block	Block	Block
Set 1	...	...	...	...

Notizen

Das ist der Sweetspot. Wenn wir einen 4-Way Set-Associative Cache bauen, brauchen wir nur vier Komparatoren. Das ist billig und kalt. Wenn unser C-Programm jetzt Variable A und B abfragt, die denselben Index haben, legen wir A in Way 0 und B in Way 1 des selben Sets. Das Thrashing-Problem ist gelöst.

Folie

55 / 81

Praxis: 2-Way und 4-Way

- Moderne CPUs nutzen typischerweise Set-Associative Caches
- Typische Größenordnungen (schematisch, je nach Generation/Produkt):
 - L1: oft 4-Way oder 8-Way
 - L3: oft höhere Assoziativität (z. B. 12–16-Way)
- Größeres N : weniger Konflikt-Misses, aber mehr Komparatoren und Strom

Angaben schematisch — konkrete Werte den jeweiligen Microarchitecture-/Datenblättern entnehmen.

Notizen

Assoziativität ist das Stellrad der Computerarchitekten. Für den L1-Cache, der extrem schnell sein muss, wählt man oft moderate Assoziativität (z. B. 4- oder 8-Way). Für den L3-Cache, bei dem Kapazität und Hit-Rate wichtiger sind als die letzte Nanosekunde, sind höhere Assoziativitäten üblich. Die konkreten Zahlen variieren stark zwischen Herstellern und Generationen.

Folie

56 / 81

Adress-Zerlegung bei Set-Associative

- Ein Set fasst mehrere Ways → insgesamt weniger Sets
- Weniger Sets → weniger Index-Bits
- Eingesparte Index-Bits wandern ins Tag
- Offset bleibt gleich (Blockgröße unverändert)

Tag (z. B. 19 Bit)	Set-Index (z. B. 7 Bit)	Offset (z. B. 6 Bit)
--------------------	-------------------------	----------------------

Notizen

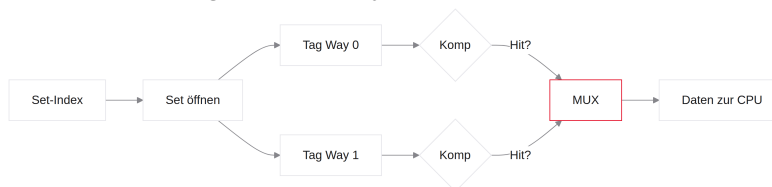
Das ist wichtig für Klausuraufgaben: Wenn Sie einen Cache von Direct Mapped auf 4-Way umbauen (bei gleicher Gesamtkapazität), schrumpft die Anzahl der Indizes um den Faktor 4. Sie brauchen also 2 Index-Bits weniger. Diese 2 Bits rücken eine Stelle nach links und werden Teil des Tags. Der Offset bleibt immer gleich, denn die Blockgröße ändert sich ja nicht.

Folie

57 / 81

Hardware-Ablauf beim N-Way Cache

1. Set-Index wählt genau ein Set
2. Alle N Ways (Tags + Valid) parallel auslesen
3. N Komparatoren vergleichen mit dem CPU-Tag
4. MUX wählt die Datenleitung des Hit-Ways



Notizen

Die Hardware arbeitet stark parallel. Wir greifen in die Schublade (das Set) und ziehen gleichzeitig vier Ordner heraus. Vier Angestellte prüfen gleichzeitig die vier Etiketten (Komparatoren). Derjenige, der das richtige Etikett hat, schiebt die Akte zur CPU durch. Die anderen drei werfen die Akten wieder zurück.

Folie

58 / 81

Eviction — Verdrängung

- Direct Mapped: voller Platz → alter Block fliegt raus (keine Wahl)
- N-Way: Set voll (alle N Ways belegt) → welcher Block wird geopfert?
- Eviction = Verdrängung
- Hardware braucht eine Replacement Policy

Notizen

Freiheit bringt Verantwortung. Wenn wir vier Plätze in unserem Set haben und alle voll sind, müssen wir eine Entscheidung treffen. Welchen der vier wertvollen Cache-Einträge opfern wir, um Platz für den neuen Block aus dem langsamen RAM zu schaffen? Wenn wir den falschen wählen, zerstören wir die zeitliche Lokalität.

Replacement: Random und FIFO

Random:

- Zufälligen Way aus dem Set werfen
- Hardware sehr einfach; bei großen Caches oft erstaunlich gut

FIFO (First-In-First-Out):

- Ältesten geladenen Block verdrängen
- Problem: früh geladene, aber oft genutzte Daten (z. B. Schleifenzähler) fliegen unnötig raus

Notizen

Die Hardware-Ingenieure haben viel experimentiert. Einfacher Zufall funktioniert erstaunlich gut und kostet fast kein Silizium. FIFO, also das Rauswerfen des ältesten Eintrags, klingt fair, ist aber oft dumm. Die Variable *i* einer Schleife wurde ganz zu Beginn geladen, wird aber in jedem Takt gebraucht. FIFO würde sie einfach rauswerfen.

Replacement: LRU

- Least Recently Used: Block mit dem ältesten Zugriff verdrängen
- Setzt zeitliche Lokalität am besten um
- Hardware-Aufwand hoch (Zeitstempel pro Way)
- Praxis oft Pseudo-LRU: Approximation mit weniger Bits

Notizen

LRU ist der König der Ersetzungsstrategien. Es wirft exakt das weg, was wir am längsten nicht mehr angeschaut haben. Das ist perfekt. Das Problem: Die CPU muss für jeden Cache-Eintrag ständig mitschreiben, wann er zuletzt benutzt wurde. Dieses ständige Aktualisieren der Zeitstempel kostet Gatter und Strom. Deshalb nutzen echte CPUs meist eine vereinfachte Approximation, genannt Pseudo-LRU.

Zusammenfassung Mapping-Strategien

Merkmal	Direct Mapped	Fully Associative	N-Way Set-Associative
Platzwahl	1 fester Platz	überall	frei innerhalb eines Sets
Komparatoren	1	sehr viele	N (z. B. 4)
Kollisionen	hoch (Thrashing)	keine	gering
Eviction	keine Wahl	LRU / Random	LRU / Random
Praxis	billige Chips	kleine Puffer (z. B. TLB)	L1/L2/L3-Standard

Notizen

Hier haben Sie alle drei Welten auf einen Blick. In der Klausur oder in der Industrie werden Sie fast immer auf Set-Associative Caches stoßen, da sie die Nachteile der beiden anderen Extreme perfekt ausbalancieren.

Fragen? — Set-Associative Mapping

- N-Way Set-Associative klar?
- Warum sinkt die Zahl der Index-Bits bei höherer Assoziativität?
- LRU nachvollziehbar?

Notizen

Haben Sie das Konzept des Set-Associative Mappings verstanden? Wenn ja, haben Sie den schwersten Teil der Computerarchitektur gemeistert. Im nächsten Schritt klären wir, was passiert, wenn wir Variablen nicht nur lesen, sondern beschreiben wollen.

Das Problem beim Schreiben

- Bisher: Lesen (l_w)
- Store (s_w): CPU schreibt einen neuen Wert
- Schreiben in den L1 ist schnell — der DRAM bleibt aber unverändert
- Cache hält dann einen neueren Wert als der RAM

Notizen

Caches machen das Lesen effizient. Beim Schreiben entsteht jedoch ein Konsistenzproblem: Wenn Sie in $C_a = 5$ schreiben, aktualisiert die CPU den Wert im L1-Cache. Der Wert im Hauptspeicher kann vorerst unverändert bleiben. Die Ebenen sind dann vorübergehend inkonsistent.

Das Cache-Kohärenz-Problem

- Asynchronität = Inkonsistenz / Verlust der Cache-Kohärenz
- Gefahr: zweiter Kern liest veralteten Wert aus dem RAM
- Gefahr: DMA schreibt veraltete Daten auf Speichermedien
- Architektur muss Updates rechtzeitig in den RAM bringen

Notizen

Solange nur ein CPU-Kern mit „seinen“ Daten arbeitet, bleibt das oft unsichtbar. Sobald ein zweiter Kern oder ein DMA-Controller auf denselben Speicher zugreift, wird die Inkonsistenz kritisch: Core 2 kann aus dem RAM noch den alten Wert lesen, obwohl Core 1 bereits aktualisiert hat. Das ist das Cache-Kohärenz-Problem — wir müssen entscheiden, wann der RAM aktualisiert wird.

Schreibstrategie 1: Write-Through

- Write-Through (Durchschreiben)
- Jeder Schreibvorgang geht synchron in L1 und Main Memory
- RAM bleibt aktuell (gegenüber dem schreibenden Kern)
- Achtung Multicore: ein anderer Kern kann trotzdem eine veraltete L1-Kopie halten — volle Kohärenz braucht Invalidierung/Snooping (Block 9)

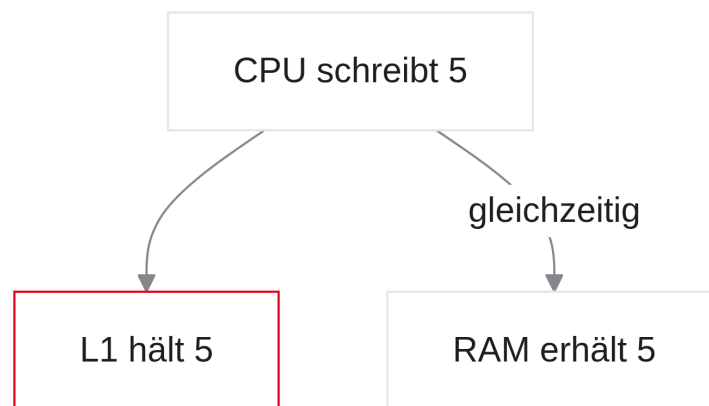
Notizen

Eine einfache Methode für die RAM-Konsistenz ist das Durchschreiben. Jeder Store-Befehl geht nicht nur in den Cache, sondern wird sofort an den langsamen RAM durchgereicht. Vorteil: keine Diskrepanz zwischen Cache und RAM beim schreibenden Kern — volle Multicore-Kohärenz braucht trotzdem Invalidierung/Snooping.

Folie

66 / 81

Write-Through: Datenfluss



Notizen

Ohne Write Buffer muss jeder Store auch den DRAM-Pfad bedienen — konsistent, aber potenziell langsam. Der Write Buffer entschärft das später.

Folie

67 / 81

Nachteile von Write-Through

- Ohne Write Buffer: jedes `sw` wartet auf den langsamen DRAM (≈ 300 Takte)
- Rückfall in die Memory Wall beim Schreiben
- Schreibintensive Programme stark ausgebremst
- Bus durch ständige Einzel-Updates überlastet

Notizen

Ohne Puffer verlieren wir den Performance-Vorteil des Caches beim Schreiben weitgehend. Wenn der C-Code eine Variable tausendmal überschreibt und jeder Store synchron auf den DRAM wartet, zahlen wir tausendmal die lange Latenz. Write-Through allein ist deshalb für performante SoCs oft zu teuer — deshalb folgt der Write Buffer.

Hardware-Hack: Write Buffer

- Write Buffer: kleine FIFO-Warteschlange zwischen L1 und RAM
- CPU schreibt rasch in Cache und Buffer und rechnet weiter
- Buffer schickt Daten im Hintergrund zum DRAM
- Wenn der Buffer voll läuft: CPU muss wieder stallen

Notizen

Der Write Buffer wirkt wie ein Postausgangskorb: Die CPU legt den neuen Wert ab und arbeitet weiter; der Controller transportiert ihn asynchron zum DRAM. Das rettet die Performance — solange der Buffer nicht überläuft. Bei dichten Speicher-Kopien kann er schnell voll sein, dann stallen wir erneut.

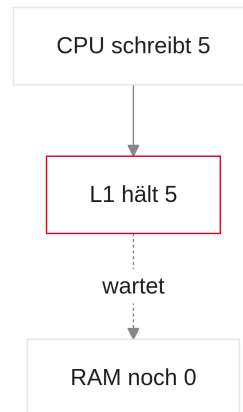
Schreibstrategie 2: Write-Back

- Write-Back (Zurückschreiben): Schreiben nur in den L1
- CPU kann dieselbe Variable tausendfach schnell aktualisieren
- Risiko: Cache ist „schmutzig“ — neuer als der RAM

Notizen

Der Industriestandard heute ist oft Write-Back. Die CPU aktualisiert Variablen im Cache, ohne bei jedem Store den RAM zu blockieren. Der Hauptspeicher kann vorübergehend veraltete Kopien halten — riskant für Kohärenz, aber vorteilhaft für die Ausführungsgeschwindigkeit.

Write-Back: Datenfluss



Notizen

Schematisch: Der neue Wert liegt im Cache; der DRAM bleibt vorerst unverändert, bis eine Eviction oder ein Kohärenzprotokoll den Block zurückschreibt.

Folie

71 / 81

Das Dirty-Bit

- Dirty-Bit (D) markiert veränderte Cache-Zeilen
- Nach Laden aus dem RAM: Dirty = 0 (sauber)
- Nach erstem `sw` in den Block: Dirty = 1

Valid	Dirty	Tag	Daten
1 Bit	1 Bit	Adress-Etikett	z. B. 64 Bytes

Notizen

Die Hardware muss den Überblick behalten. Neben dem Valid-Bit bekommt jede Zeile ein Dirty-Bit. Ein schmutziger Block bedeutet: „Achtung, meine Daten sind wertvoller als die im RAM. Verliere mich nicht!“

Eviction bei Write-Back

- Daten gelangen erst bei Verdrängung in den Main Memory
- Bei Ersetzen von Block A durch Block B: Dirty-Bit von A prüfen
- Dirty = 0: A einfach verwerfen
- Dirty = 1: A zuerst zurückschreiben, dann B laden
- Folge: Miss-Penalty kann sich verdoppeln (Write-Back + Read)

Notizen

Das ist der Moment der Wahrheit. Erst wenn ein Block aus Platzmangel aus dem Cache geworfen werden muss, wird die Rechnung beglichen. Ist der Block schmutzig, muss ihn der Memory Controller erst mühsam in den RAM zurückschreiben, bevor der neue Block einziehen darf. Das verursacht bei einem Cache-Miss plötzlich die doppelte Miss-Penalty (erst rausschreiben, dann neu lesen), was für Performance-Analysen sehr relevant ist.

Vergleich Schreibstrategien

Merkmal	Write-Through	Write-Back
RAM-Update	sofort	erst bei Eviction
RAM aktuell	ja	oft veraltet (Dirty)
Performance	schlecht (ohne Buffer)	hervorragend
Bus-Last	hoch	gering
Hardware	gering	Dirty-Bits nötig

Notizen

Hier der klassische Vergleich für die Klausur. Write-Through ist simpel und sicher, aber verstopft den Bus. Write-Back ist die Königsklasse für Geschwindigkeit, erkaufte sich das aber durch Hardware-Aufwand und komplexe Cache-Kohärenz-Protokolle, wenn wir Multicore-Systeme in Block 9 betrachten.

Software optimieren

- Hardware bekannt: Cache Lines (räumlich) und LRU (zeitlich)
- Gute Nachricht: Caches müssen in C nicht manuell programmiert werden
- Schlechte Nachricht: gegen die Hardware zu programmieren kostet leicht Faktor 100
- Software-Engineering heute: C-Code für maximale Hits, minimale Misses

Notizen

Damit verlassen wir die Gatter-Ebene und setzen den Hut des C-Programmierers auf. Das wertvollste Wissen aus dieser Vorlesung für Ihren Berufsalltag: Schreiben Sie cache-freundlichen Code. Wenn Sie wissen, dass der Cache 64-Byte-Blöcke lädt, dürfen Sie niemals wild kreuz und quer durch den Speicher springen.

Labor-Szenario: 2D-Arrays

- Beispiel: Matrizen, Bilder, Tabellen
- Bild mit 1000×1000 Pixeln auf 0 setzen
- Variante A oder Variante B — mathematisch gleich, architektonisch nicht

```
int bild[1000][1000];  
  
// Variante A oder Variante B?
```

Notizen

Genau dieses Phänomen eignet sich gut für ein späteres Labor: Eine Matrix auf Null setzen — zeilenweise oder spaltenweise. Mathematisch identisch, architektonisch oft stark unterschiedlich in der Miss Rate.

2D-Arrays im RAM — Row-Major Order

- RAM ist linear, kein 2D-Gitter
- C speichert zeilenweise (Row-Major Order)
- `[0][0]`, `[0][1]`, `[0][2]` liegen nebeneinander
- `[0][0]` und `[1][0]` liegen 1000 Integers (≈ 4000 Bytes) auseinander



Notizen

Das ist der Schlüssel. Der C-Compiler legt mehrdimensionale Arrays flach aus. Die erste Zeile liegt im RAM am Stück. Danach kommt direkt die zweite Zeile. Das bedeutet: Wenn Sie horizontal durch die Zeile gehen, spazieren Sie physikalisch zu direkten Nachbarn. Wenn Sie vertikal (von Zeile 0 in Zeile 1) springen, überspringen Sie physikalisch tausende von Bytes!

Cache-Albtraum: spaltenweiser Zugriff

```
// Column-Major Traversal - schlecht für den Cache
for (int col = 0; col < 1000; col++) {
    for (int row = 0; row < 1000; row++) {
        bild[row][col] = 0;
    }
}
```

Innere Schleife erhöht `row`: Zugriff `[0][0]`, `[1][0]`, `[2][0]`, ...

Notizen

Schauen Sie sich diesen Code an. Die innere Schleife erhöht `row`. Das heißt, wir gehen spaltenweise nach unten. Mathematisch ist die Matrix am Ende genullt. Aber schauen wir uns an, was der Cache dabei durchmacht.

Was passiert beim spaltenweisen Zugriff?

- `[0][0]`: Miss — Line lädt z. B. `[0][0] ... [0][15]`
- `[1][0]`: ≈ 4000 Bytes weiter — wieder Miss
- `[2][0]`: wieder Miss
- Mitgebrachte Nachbarn bleiben ungenutzt und werden verdrängt
- Resultat: Miss Rate nahe 100 %

Notizen

Das ist die Definition von Thrashing durch schlechte Software. Der Cache ist so freundlich und holt 15 Nachbarn mit, weil er auf räumliche Lokalität wettet. Und was machen wir? Wir ignorieren die Nachbarn, springen 4000 Bytes weiter und zwingen den Cache zu einem neuen Miss. Wir verletzen das Lokitätsprinzip. Die Ausführungszeit explodiert.

Cache-Traum: zeilenweiser Zugriff

Nur die Schleifen vertauschen:

```
// Row-Major Traversal - cache-freundlich
for (int row = 0; row < 1000; row++) {
    for (int col = 0; col < 1000; col++) {
        bild[row][col] = 0;
    }
}
```

- `[0][0]`: Miss — Line lädt Nachbarn
- `[0][1]`, `[0][2]`, ...: Hits (nur Hit Time)
- Minimale Miss Rate — oft um ein Vielfaches schneller

Notizen

Zwei Schleifen vertauscht, gleiche Mathematik — aber oft deutlich schneller. Wir gehen zeilenweise durch. Die innere Schleife rückt zum physikalischen Nachbarn. Ein Miss erkaufte typischerweise mehrere Hits, die nur noch die Hit Time kosten. Genau so entsteht cache-freundlicher Code für Bildverarbeitung oder lineare Algebra.

Ausblick: Visualisierung im Labor

- Geplantes Laborziel: denselben Effekt messbar machen
- Werkzeugkandidat: Ripes (Data-Cache-Ansicht) — Details folgen mit dem Übungsblatt
- Matrix-Code: Hits/Misses beobachten
- Assoziativität und Schreibstrategie vergleichen

Notizen

Die Theorie endet hier bewusst mit einem Ausblick. Sobald das Übungsblatt steht, können Sie Hit-Raten messen und z. B. Direct Mapped gegen 4-Way vergleichen. Bis dahin reicht es, Row-Major vs. Column-Major und die Mapping-Konzepte sicher zu beherrschen.

Zusammenfassung Block 2

- Memory Wall erzwingt Speicherhierarchie
- Caches basieren auf Lokalität (zeitlich und räumlich)
- $AMAT = Hit\ Time + Miss\ Rate \times Miss\ Penalty$
- Direct Mapped: schnell, aber Thrashing-anfällig
- Set-Associative: Kollisionen innerhalb eines Sets entschärfen
- Write-Through / Write-Back steuern RAM-Kohärenz
- Software: cache-freundliche Strukturen (z. B. Row-Major Traversal)

Notizen

Wir haben damit begründet, warum CPUs trotz relativ langsamem Hauptspeicher leistungsfähig bleiben: Der Cache nutzt Lokalität aus. Im nächsten Block folgen Interrupts und System-Timer. Vielen Dank — und viel Erfolg beim Üben der Konzepte.